

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Lending Protocol	Documentation quality	Medium	
Timeline	2024-06-10 through 2024-07-12	Test quality	Medium	
Language	FunC	Total Findings	32	Fixed: 24 Acknowledged: 7 Mitigated: 1
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review	High severity findings ⓘ	1	Mitigated: 1
Specification	Official doc ⓘ	Medium severity findings ⓘ	5	Fixed: 3 Acknowledged: 2
Source Code	evaafi/contracts ⓘ #bc92f2b ⓘ	Low severity findings ⓘ	5	Fixed: 5
Auditors	- Valerian Callens Senior Auditing Engineer - Julio Aguilar Auditing Engineer - Gereon Mendler Auditing Engineer	Undetermined severity findings ⓘ	14	Fixed: 12 Acknowledged: 2
		Informational findings ⓘ	7	Fixed: 4 Acknowledged: 3

Summary of Findings

EVAA is a decentralized lending protocol designed for the TON ecosystem. It allows users to deposit and borrow assets and offers dynamic interest rates based on supply and demand. If collateral falls short, liquidations can protect the system from insolvency. There are two main contracts. The master contract is a pool of multiple assets and a central hub for all system interactions. It orchestrates and relays user requests to the user contract that handles deposits, withdrawals, borrowing, repayments, and liquidations.

The code is well-written, and the test suite covers several happy and unhappy paths. The quality of the documentation is good with both textual descriptions and visual flows. However, it can be outdated for some specific components and it is not always clear to which version of the codebase a specific part of the documentation refers. Also, the test suite still contains some "todo" statements. We identified several issues that need to be resolved before deploying this version of the codebase.

The EVAA team was highly engaged and responsive throughout the audit, promptly addressing our questions and participating in productive discussions.

Fix review update

The fix review reviewed the commit `55096cf1fd091629ff8dad783f71fb4758eded46`. The EVAA team actively addressed all issues and suggestions (Fixed, Mitigated, or Acknowledged). Also, they self-identified 7 issues ([EVAA-26](#) to [EVAA-32](#)) during the audit and included their resolution during the fix review phase. The codebase is now more clear and robust. The flow to supply assets to the system has been simplified and more events are now logged, which can help off-chain observers to track what happens on-chain more easily. Also, tests were added to cover the updates. Finally, we strongly recommend explicitly documenting to the users the key actors of the system and their capabilities.

ID	DESCRIPTION	SEVERITY	STATUS
EVAA-1	Function <code>update_full_config_process()</code> and Debug Code Gives Excessive Power to the Admin	• High ⓘ	Mitigated
EVAA-2	Risk of Uncaught Unexpected Errors on Multi-Contract Flows with No Rollback of Latest State Updates Can Lead to Inconsistent State Storages or Funds Locked	• Medium ⓘ	Fixed

ID	DESCRIPTION	SEVERITY	STATUS
EVAA-3	Missing Input Validations	● Medium ⓘ	Fixed
EVAA-4	Possible Unfair Race Conditions Between Borrowers and Liquidators when the System Is Activated Again	● Medium ⓘ	Acknowledged
EVAA-5	Users with Liquidatable Positions Might Be Blocked From Repaying	● Medium ⓘ	Acknowledged
EVAA-6	Incorrect Amount Returned by <code>calculate_maximum_withdraw_amount()</code>	● Medium ⓘ	Fixed
EVAA-7	[False Positive]	● Low ⓘ	Fixed
EVAA-8	Incorrect Message Mode Lets Anyone Drain Available Native Tokens From Any User Contract	● Low ⓘ	Fixed
EVAA-9	Event Management Could Be Improved	● Low ⓘ	Fixed
EVAA-10	Suggestion to Add Invariant Checks to Fail if Unexpected Edge Cases Happen	● Low ⓘ	Fixed
EVAA-11	Unclear Usage of Variable <code>dust</code>	● Low ⓘ	Fixed
EVAA-12	Concurrent Operations Exposed to Race Conditions	● Informational ⓘ	Acknowledged
EVAA-13	Any Address Can Lock for a Short Period the User Contract of Another User by Supplying a Minimum Amount on Its Behalf	● Informational ⓘ	Fixed
EVAA-14	Recommended Usage of <code>end_parse()</code>	● Informational ⓘ	Fixed
EVAA-15	Getters Do Not Return Information About the Validity of <code>prices_packed</code>	● Informational ⓘ	Acknowledged
EVAA-16	Unclear Fee Management	● Informational ⓘ	Fixed
EVAA-17	General Risks Related to Using Price Oracles	● Informational ⓘ	Acknowledged
EVAA-18	Mismatch Between Comments in Liquidation and Supply Functions	● Informational ⓘ	Fixed
EVAA-19	Impact of Deprecating Assets	● Undetermined ⓘ	Acknowledged
EVAA-20	Each Muldiv Operation Could Explicitly Force a Round-up or Round-Down Based on the Context to Make the Behavior of the Code More Explicit and Controlled	● Undetermined ⓘ	Fixed
EVAA-21	Usage of <code>impure</code> Specifier	● Undetermined ⓘ	Fixed
EVAA-22	Supply and Borrow Rates Might Be Outdated in Multiple Situations	● Undetermined ⓘ	Fixed
EVAA-23	Missing Config Validation After Code Upgrade	● Undetermined ⓘ	Fixed
EVAA-24	Production Readiness of the Codebase	● Undetermined ⓘ	Fixed
EVAA-25	Liquidators Are Incentivized to Perform More Smaller Liquidations to Get More Collateral Rewards	● Undetermined ⓘ	Acknowledged

ID	DESCRIPTION	SEVERITY	STATUS
EVAA-26	[Self-Identified] Tracking Indexes Calculated Based on New Total Supply and Borrow	• Undetermined ⓘ	Fixed
EVAA-27	[Self-Identified] Repayments Can Be Blocked Due to the Current Max Cap Check	• Undetermined ⓘ	Fixed
EVAA-28	[Self-Identified] Global Tracking Indexes Not Updated when Updating Tracking Speed Config Parameters	• Undetermined ⓘ	Fixed
EVAA-29	[Self-Identified] Insufficient Validations of Messages Received From <code>user.sc</code>	• Undetermined ⓘ	Fixed
EVAA-30	[Self-Identified] Incorrect Message Body Parsed when Dealing with Reverted Operations	• Undetermined ⓘ	Fixed
EVAA-31	[Self-Identified] Adding New Tokens Reverts Due to Incorrect Sequence of Operations	• Undetermined ⓘ	Fixed
EVAA-32	[Self-Identified] Supply Cap Check Can Be Bypassed	• Undetermined ⓘ	Fixed

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

i **Disclaimer**

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
 1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
 2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
 3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
 1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
 2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

The scope of this audit includes all files in the repository `https://github.com/evaafi/contracts` at the commit `bc92f2bc31a4565df3d6930b71699e45e997667f` in the folder `contracts/*`, except the file `contracts/external/stdlib.fc`.

Operational Considerations

- We recommend using a multi-sig contract for admin actions and following best practices for key management.
- The protocol parameters for collateral and liquidation management should be selected wisely to offer sufficient incentives to liquidators and prevent insolvency.
- Also, this audit cannot guarantee the behavior of any future system update. In the TON ecosystem, contracts are not necessarily immutable, as FunC allows for upgradeability. Both the master and user contracts possess this capability. This audit only examined a specific version of the codebase. Any future code update should be properly audited and users should be notified before the update happens. While upgradability is not inherently a vulnerability, users should be aware that the contracts can be upgraded at any time.

Key Actors And Their Capabilities

- The admin of the `master` contract has several key abilities. It can update both the master and user contract code, modify the entire storage layout of the master contract using the `init_master_process()` and `update_full_config_process()` functions, and adjust any user's principals through the `debug_principals_edit_master_process()` function. The admin can also claim reserve assets for the EVAA team, update the asset dynamics collection (which includes total supplied, borrowed, and balance amounts as well as supply and borrowing rates), pause the master contract to prevent users from depositing, withdrawing, or liquidating, and add new assets to the asset dynamics collection.
- On the other hand, the user is the owner of their `user` contract. While users cannot interact directly with their contract, they can manage their positions through the master contract. It allows them to deposit, withdraw, borrow, and repay funds. Additionally, they can liquidate any user position that is liquidable.

Fix review update

The admin of the `master` contract has less key abilities, and debug functions were removed as a fix of [EVAA-1](#). In order to be able to react in case of emergency, a multisig contract has been added with extensive capabilities. According to the EVAA team, this multisig is expected to have a threshold of 3/4, and only one signer will be owned by the EVAA team, the rest being owned by other people from the TON ecosystem to avoid privilege abuses.

Findings

EVAA-1

Function `update_full_config_process()` and Debug Code Gives Excessive Power to the Admin • High ⓘ Mitigated

i Update

The debug functions were removed and the remaining privileged actions were split between the existing admin and a more powerful multisig. The audit team recommends that the existence of this multisig and the signers should be documented to end users. Also, in order to reduce the risk of manual errors and additional efforts to fix them, we recommend to prevent the multisig to send a message with a mode that will destroy the contract, even if it can be redeployed later at the same address.

i Update

Marked as "Mitigated" by the client.
Addressed in: `fe6420e679701fdcad696cc5f36955a87d078c48`, `78166aa73e1e66ab93bf94b7564600780903b66e`.
The client provided the following explanation:

We have minimized the admin wallet permissions. We added additional checks on already existing admin functions (that the wallet admin can't break the contract vault). But we decided to create a hard backdoor in case something goes wrong. The hard backdoor is controlled by the multisig wallet `EQDPc2bgQSXwXfjW5H7ZZQjcIX2K-tnfCStMjr8DxabL1geU`, which will be owned by a committee of trusted individuals: three signatures are trusted individuals outside of EVAA, one is controlled by EVAA. The threshold for the multisig is three signatures.

File(s) affected: `master.fc`, `master-admin.fc`, `user.fc`, `user-admin.fc`

Description: The function `master-admin::update_full_config_process()` replaces the entire master storage, including:

- `asset_dynamic_collections` , which should not be externally modified. This dictionary holds crucial accounting variables for each whitelisted asset, updated internally through user interactions like supply, borrow, and liquidation. These variables determine withdrawal limits, debt, and yield growth. External changes to this dictionary could prevent users from withdrawing funds or disrupt the system's accounting.
- `new_upgrade_info` , which can be used to bypass the upgrade process of both contracts `master` and `user` .

We also want to highlight the existence of debug code in both contracts `master` and `user` that allows the admin to modify `asset_dynamic_collections` (supply and borrow rates as well as total supplied, total borrowed and token balances) and adjust users' principals, including resetting debt or supplied amounts.

Recommendation: Consider removing or adapting these functions to reduce the excessive power the admin has over the code of the system and the system's and users' balances.

EVAA-2

Risk of Uncaught Unexpected Errors on Multi-Contract Flows with No Rollback of Latest State Updates Can Lead to Inconsistent State Storages or Funds Locked

• Medium ⓘ Fixed

✓ Update

The code was updated to avoid or handle exceptions.

✓ Update

Marked as "Fixed" by the client.

Addressed in: 8196fb0bb4832d642f651b5acb1988506a0ce052 , b70f51022b6caca548d807a571c679053f1d5229 .

Description: There are three multi-contract flows resulting in state updates that may not be reverted in case of unexpected errors. This could occur at any step except for the initial one.

For the supply flow, 1) assets are transferred to the master contract, 2) the user contract is locked, 3) the master contract verifies if the max supply token is reached, if yes, saves the new balance and 4) sends a success or failure message to the user contract that can update the rewards and accrue the tracking indexes. Here, an unexpected error can happen:

- in step 2) if `s_rate == 0` or `b_rate == 0` , which will create a divide by `0` error in `principal_value_supply_calc()` or `principal_value_supply_calc()`
- in step 3) if `min_principal_for_rewards == 0` and `new_total_supply == 0` or `new_total_borrow == 0` , which will create a divide by 0 error in `supply_success_process()` .
- in step 3) if the value of `forward_ton_amount` is higher than the amount of TON available in the contract.
- in step 4) if `s_rate == 0` or `b_rate == 0` , which will create a divide by `0` error in `principal_value_supply_calc()` or `principal_value_supply_calc()` . If any of these cases occur, state updates of previous steps will not be reverted, with a possible consequence of assets locked, the user contract remaining in a locked state, the master contract with a new balance even if in the user contract, the rewards are not considered and the tracking indexes are not updated.

For the withdrawal flow, 1) the request is registered in the master contract, 2) the user contract is locked, 3) the correct amount is withdrawn from the balance of the master contract which 4) sends a success or failure message to the user contract that can update the rewards and accrue the tracking indexes. Here, an unexpected error can happen:

- at several steps if `s_rate == 0` or `b_rate == 0` , which will create a divide by `0` error in `principal_value_supply_calc()` or `principal_value_supply_calc()` .
- in step 3) if the value of `forward_ton_amount` is higher than the amount of TON available in the contract. If it occurs, here again, state updates of previous steps will not be reverted, leaving the contracts in an inconsistent state.

For the liquidation flow, 1) assets are transferred to the master contract, 2) the user contract is locked for liquidation, and the exact amounts are calculated as well as the principals are updated, 3) the master contract updates its accounting variables and 4) sends a success or failure message to the user contract that can update the rewards and accrue the tracking indexes. Here, an unexpected error can happen:

- in each step where `s_rate == 0` or `b_rate == 0` and the functions `principal_value_supply_calc()` or `principal_value_supply_calc()` are executed, which will create a divide by `0` error.
- in step 2), if `collateral_price == 0` in `get_collateral_quote()` or `liquidate_user_process()` , which will create a divide by `0` error.
- in step 2), if `borrow_amount == 0` in `liquidate_user_process()` , which will create a divide by `0` error.
- in step 3), if `new_collateral_total_supply == 0` or `new_collateral_total_borrow == 0` or `loan_new_total_supply == 0` or `loan_new_total_borrow == 0` in `liquidate_satisfied_process()` , which will create a divide by `0` error.

While these cases are unlikely with the current codebase (in particular, `s_rate == 0` or `b_rate == 0`), it is better to make sure that explicit errors help users to throw properly and do all necessary state rollbacks so that the system does not remain in an inconsistent state.

Recommendation: Consider making sure that explicit errors let user contracts throw properly and do all necessary state rollbacks so that the system does not remain in an inconsistent state.

EVAA-3 Missing Input Validations

• Medium ⓘ Fixed



Update

All items were fixed or considered.



Update

Marked as "Fixed" by the client.

Addressed in: 246f5a27675ad3a593a4385f8087fdaf98051a51 .

File(s) affected: master-admin.fc, master-liquidate.fc

Description: It is important to validate inputs, even if they only come from trusted addresses, to avoid human error that would result in breaking core system invariants:

1. For each asset, the invariant `0 <= collateral_factor <= liquidation_threshold <= 10000` should be preserved;
2. The value of `liquidation_bonus` should be greater than `10000` .
3. The value of `liquidation_reserve_factor` should be lower than `constants::reserve_liquidatoin_scale` ;
4. The value of `reserve_factor` should be lower than `constants::reserve_scale` ;
5. It is possible with the function `add_new_token_dynamics_process()` to override an existing supported token, even if the name of the function suggests the opposite.
6. It may be relevant to add input validation for the values of `borrow_rate_slope_low` , `borrow_rate_slope_high` , `supply_rate_slope_low` , `supply_rate_slope_high` , `target_utilization` , `origination_fee` .
7. It is possible for the admin via the operation `op::update_full_config` to override the current storage of the `master` contract without any integrity checks about the old or new variables. This manual operation can be prone to manual error and integrity checks could be added to make sure that the storage cannot be updated into an inconsistent state. For instance, this lack of validation could theoretically disrupt the local storage layout and, therefore, the system accounting, potentially resulting in users losing their funds.
8. In `master-liquidate.fc` , add a check to revert if a liquidator uses the same `asset_id` for the collateral and the repaid assets.

Recommendation: Consider adding the relevant checks.

EVAA-4

Possible Unfair Race Conditions Between Borrowers and Liquidators when the System Is Activated Again

• Medium ⓘ Acknowledged



Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

We mitigate this on the process level: before update we warn in advance users to take action for securing their positions against volatility

File(s) affected: master-storage.fc

Description: It is possible for the admin via the functions `update_dynamics_process()` , `disable_contract_for_upgrade_process()` , `force_enable_process()` and `cancel_upgrade_process()` to disable or enable the master contract. If the protocol is temporarily disabled with active positions, these positions could become liquidable and, when the protocol is set back to active, a race condition will happen between borrowers who want to send back more collateral and liquidators who want to liquidate these users, which seems unfair for borrowers who were not able to do that earlier because protocol operations were disabled.

Recommendation: Consider adding a period when the system is activated again to delay the liquidations but make it possible instantly to supply more collateral to favor depositors over liquidators.

EVAA-5

Users with Liquidatable Positions Might Be Blocked From Repaying

• Medium ⓘ Acknowledged



Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

We haven't fix it since it involves complete rebuilding of the system with relatively low benefits.

File(s) affected: `user-liquidate.fc`

Description: When a liquidation begins, the state of the user's contract is modified by incrementing its value by one. This action blocks the contract owner from making deposits or withdrawals while allowing others to continue liquidating the user's position. In a highly volatile market, it is possible that the owner could attempt to deposit funds to save their position, but a liquidator might act more quickly and complete the liquidation first while also permitting other users to liquidate blocking the owner from keeping the position.

Recommendation: We recommend allowing deposits and liquidations at the “same time”. In this way, the owner of the user contract can minimize the loss in such scenarios.

EVAA-6 Incorrect Amount Returned by `calculate_maximum_withdraw_amount()`

• Medium ⓘ

Fixed

- ✓ Update

The function `get_available_to_borrow()` is now used to get the aggregated collateralization of the user, while keeping a maximum withdrawable amount being `old_present_value` , such that the user cannot withdraw more than what he supplied for this token.
- ✓ Update

Marked as "Fixed" by the client.
Addressed in: `20e65ff5839682334207e591427b81e7b397e1f8` .

File(s) affected: `user-utils.fc`

Description: A different collateral factor can be used for two different assets. Since in `user-utils::calculate_maximum_withdraw_amount()` the collateral factor is used on aggregated balances (taking into account several assets), issues could happen if different Collateral Factors are set as described below: The issue resides in the formula to calculate `max_amount_to_reclaim` . Using a scenario of supplying `100 USD` of A, `100 USD` of B, and borrowing `65 USD` of C:

- `CF A = 65%` , `CF B = 60%` ;
- aggregated supply balance is `200 USD` ;
- aggregated borrow balance is `65 USD` ;
- total amount of C that can be borrowed is `65+60 = 125 USD`
- max amount to reclaim for A is `200 - (65/0.65) = 100` .
- `100 USD` of A is available so the function says the user can withdraw `100 USD` of A. After the withdrawal, the supply is only `100 USD` of B. Since `CF B is 60%` , the user cannot borrow more than `60 USD` of C. But he has `65 USD` of C. So the function `user-utils::calculate_maximum_withdraw_amount()` returned an amount to withdraw that is too high, because it would make the user over-collateralized.

Recommendation: Consider making sure to correctly calculate the aggregated collateralization of a user by multiplying the correct amount of a given asset with its associated collateral factor.

EVAA-7 [False Positive]

• Low ⓘ

Fixed

Description: This issue, reported in the initial report, appeared to be a false positive.

EVAA-8 Incorrect Message Mode Lets Anyone Drain Available Native Tokens From Any User Contract

• Low ⓘ

Fixed

- ✓ Update

Fixed as recommended.
- ✓ Update

Marked as "Fixed" by the client.
Addressed in: `489b5678a8ced064b46de4e852a175b26aecee05` .

File(s) affected: `user.fc`

Description: Any address can send an internal message to user sc with `op::get_store` . As a result, the contract returns the current storage of the contract, and all the balance minus a few TONS reserved for the storage, using the mode `sendmode::CARRY_ALL_BALANCE + 2` . It could be assumed that only the user of the lending wallet should be able to do this operation, or that at least not all the exceeding TONS are sent to an arbitrary address. The severity is Low since no significant balance is expected in the user contracts.

Recommendation: Consider updating the mode to `sendmode::CARRY_ALL_REMAINING_MESSAGE_VALUE`.

EVAA-9 Event Management Could Be Improved

• Low ⓘ Fixed

✓ Update

11 additional operations are logged (details in `constants/logs.fc`).

✓ Update

Marked as "Fixed" by the client.
Addressed in: `ee06ab7aa984bc704426ad011c46de73ea1375b5`.

Description: Only three operations are logged: supply, withdrawal, and liquidation. However, other operations are not logged such as enabling or disabling the master contract, or updating the storage of the master contract (`op::update_dynamics`, `op::update_full_config`). Also, tracking the operations `op::claim_asset_reserves` could help to reconstruct via event monitoring the current accounting of the protocol. This lack of information or events could make harder the monitoring of on-chain operations by off-chain observers such as liquidators or risk management tools. It could also make the identification of a hack attempt unnoticed (ex: updating the oracles parameters).

Recommendation: Consider improving the coverage of core system operations by events wherever relevant.

EVAA-10

Suggestion to Add Invariant Checks to Fail if Unexpected Edge Cases Happen

• Low ⓘ Fixed

✓ Update

All items were fixed or considered.

✓ Update

Marked as "Fixed" by the client.
Addressed in: `6ee0dfe3f579deecab72516ffe84dd0821dd24ec` and `51bf954e0c5020e2ebdcb88289098ccc93a43204`.

File(s) affected: `utils.fc`, `user-utils.fc`, `user-withdrawal.fc`, `master-admin.fc`, `asset-dynamics-packer.fc`

Description: At several locations in the code, invariant checks may be desired to make sure that unexpected edge cases cannot happen. For instance:

1. In `user-utils::calculate_maximum_withdraw_amount()`, a check could return `0` or throw if `get_avaliable_to_borrow()` returns a negative value.
2. In `user-withdrawal::withdraw_user_process()`, the if condition `if (borrow_is_collateralized)` could become `if (enough_price_data & borrow_is_collateralized)` to explicitly only accept this case and do nothing otherwise. If not, updating the code of the called function in future versions may change the assumptions at the level of the calling function, which is the case here.
3. In `utils::around_zero_split()`, we should check that `lower <= upper`.
4. In `master-admin::submit_upgrade_process()`, the function should revert if `new_master_code` and `new_user_code` are both `null`.

The next item was self-identified by the EVAA team during the audit and was added to the report.

5. In `asset-dynamics-packer::pack_asset_dynamics()`, the function should revert if any of `total_supply_principal` or `total_supply_principal` is negative.

Recommendation: Consider checking if relevant core invariants could be added to the code and add them, while making sure that it cannot become a way to DoS the system or to throw without rolling back the state updated during previous contract interactions within a multi-contract flow.

EVAA-11 Unclear Usage of Variable `dust`

• Low ⓘ Fixed

✓ Update

Both checks are now done with principals.

✓ **Update**

Marked as "Fixed" by the client.
Addressed in: `a9a8e0282c7dcad513d7c8923ff790d4ac82dc98` .

File(s) affected: `master-supply.fc`, `user-utils.fc`

Description: The variable `dust` is used within the system to simplify calculations and give a smoother user experience (getting rid of residuals). However, it is compared with two types of values:

- principals in `master-supply::supply_success_process()` ;
- present values in `user-utils::calculate_maximum_withdraw_amount()` ;

Recommendation: Consider clarifying with which type of value this variable should be compared.

EVAA-12 Concurrent Operations Exposed to Race Conditions

• **Informational** ⓘ **Acknowledged**

ⓘ **Update**

Marked as "Acknowledged" by the client.
The client provided the following explanation:

```
Repay VS liquidate is already mentioned in EVAA-5
withdraw VS idle contract is not issue at all, we need idling only for updating user sc (for example
if we find any issue in user sc we may manually idle all user scs to get rid of exploit, without
waiting for any supply/withdraw/liquidate)
supply VS supply is fixed in EVAA-13
```

Description: Race conditions can exist in the system and some concurrent operations in the code could be impacted by this. For instance:

- repay VS liquidate;
- withdraw VS idle contract;
- withdraw VS withdraw due to limited available assets;
- supply VS supply due to the max supply cap of a given asset;

Recommendation: Consider identifying impacted functions and risks should be documented to users.

EVAA-13 Any Address Can Lock for a Short Period the User Contract of Another User by Supplying a Minimum Amount on Its Behalf

• **Informational** ⓘ **Fixed**

✓ **Update**

The flow to supply assets has been updated such that locking the state of user contracts is no longer required.

✓ **Update**

Marked as "Fixed" by the client.
Addressed in: `e1428a6ff80f471eef937338c68a90e9f5a92a23` , `36f6e259d8536fb49602df716baee0ac8c205b1c` ,
`dce63d9f6d10e29112424d8424cff5c8c0820811` .

Description: Any address can lock for a short period the user contract of another user by supplying a minimum amount on its behalf. No clear negative scenario has been identified.

Recommendation: Consider assessing if this should be the expected behavior.

EVAA-14 Recommended Usage of `end_parse()`

• **Informational** ⓘ **Fixed**

✓ **Update**

The function `end_parse()` is now used where relevant.

✓ Update

Marked as "Fixed" by the client.

Addressed in: `3b90e350ac4cf531b83f51ecacab7ccb139bc886` , `2e3e14b148beb44555b7924d6e48a0d610689d9b` .

File(s) affected: `user-storage.fc`, `master-storage.fc`

Description: The official doc of FunC recommends as follows:

Use ``end_parse()`` wherever possible **when** reading data **from storage and** the message payload. Since TON uses `bit` streams **with** variable data **format**, it's helpful **to** ensure that you **read as** much **as** you **write**. This can save you an hour **of** debugging."

Some eligible areas of the codebase are not following this recommendation, like for instance in `user-storage.fc` or `master-storage.fc` .

Recommendation: Consider using `end_parse()` wherever relevant, but where it should not introduce a potential source of Denial Of Service.

EVAA-15

Getters Do Not Return Information About the Validity of

• Informational ⓘ

Acknowledged

`prices_packed`

i Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

Getters can be invoked by users only (no contract-to-contract invocation). bearing in mind we use on-demand oracle model, user should provide prices himself and care about price validity

File(s) affected: `user-get-methods.fc`

Description: The following getters functions must be called with a valid cell `prices_packed` to return information about the user smart contract. However, the fact that the cell `prices_packed` contained invalid data is ignored by the following functions:

1. `user-get-methods::getAccountHealth()` ;
2. `user-get-methods::getAvailableToBorrow()` ;
3. `user-get-methods::getIsLiquidable()` ;
4. `user-get-methods::getAggregatedBalances()` ;
5. `user-get-methods::get_maximum_withdraw_amount()` ;
6. `master-get-methods::getCollateralQuote()` ;

As a result, users will not know if the value returned by these functions is correct. Also, these functions cannot be used to verify if the prices provided by oracles are valid for the system, which is even more relevant for liquidation bots that may want to use these getters to make periodical validity checks of oracle data.

Recommendation: Consider returning two values instead of one: the returned value and the validity of `prices_packed` .

EVAA-16 Unclear Fee Management

• Informational ⓘ

Fixed

✓ Update

Listed issues were either fixed or clarified as working as intended.

✓ Update

Marked as "Fixed" by the client.

Addressed in: `2e2a8b1b7727a8782b2b5a7dded07c4425f4d7b2` .

Description: The following is still unclear:

1. Fees for logs (`fee::log_tx`) are deduced from `msg_value` in `master_liquidate.fc` and `master_withdrawal.fc` , but not in `master_supply.fc` .
2. Fees for forwarding a cell (`cell_fwd_fee()`) are included only in `withdraw_min_attachment()` but not in similar functions for the supply and liquidate flows.

Recommendation: Consider clarifying if this works as expected or not.

EVAA-17 General Risks Related to Using Price Oracles

• Informational ⓘ Acknowledged

Update

Marked as "Acknowledged" by the client.

Description: The system uses prices used by whitelisted oracles to calculate the current amounts of assets used by the protocol in specific denominations. This leads to several risks:

1. Extreme prices can be provided due to an error, a price manipulation attack, or extreme market conditions;
2. The number of decimals used can differ between different oracles;
3. An oracle can become unavailable. If the number of available oracles goes below the current threshold, no more operations using prices can work until these oracles are available again.

Recommendation: Consider assessing these risks and defining measures to reduce the likelihood of these scenarios.

EVAA-18

Mismatch Between Comments in Liquidation and Supply Functions

• Informational ⓘ Fixed

Update

Listed issues were clarified.

Update

Marked as "Fixed" by the client.
Addressed in: `e89ff56b8149ec9c3230eeb9099f6fb28c7f8022` .

File(s) affected: `master-supply.fc`, `master-liquidate.fc`

Description: In the function `master-liquidate::master_core_logic_liquidate_asset_unchecked()` , we have the following comment:

```
;; note that the asset balance does NOT increase at this point (unlike in Supply)
;; because Supply always succeeds, the received assets might be used in other operations immediately
;; but liquidate might not succeed – in which case it should refund the assets – thus these assets can't
be made available yet
```

However, in the function `master-supply::master_core_logic_supply_asset_unchecked()` , we have the following comment:

```
;; got money, but we don't update token_balance yet
;; that is because supply_fail is possible,
;; so we might need to refund money-received later
;; and we gotta make sure this money is available
;; (so they can't be used meanwhile until supply_success is confirmed)
```

These two comments are contradictory.

Finally, a third comment is incorrect, in `supply_success_process()` , when the token supply cap is reached:

```
;;revert principals on user sc
```

Recommendation: Consider aligning the comments and the code for all three items.

EVAA-19

Impact of Deprecating Assets

• Undetermined ⓘ Acknowledged

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

```
We do not removing assets from our config, our system is not designed for it (at least in this
version).
Our process of deprecating assets based on economical incentives only. It looks as follows:
0) we notify users about deprecation
```

1) we set `reserve_factor` to 100%. This leads to 0% supply apy and deposit attrition

2) we set `collateral_factor` to 0. This mitigates borrowers (that use the asset as collateral) inflow

As a result of above mentioned most of all suppliers withdraw their funds which leads to 100% utilisation and borrow rate spike. Spike in rates motivates borrowers to repay debt and in a while protocol gets rid of almost all borrows and supplies of the asset.

File(s) affected: `user-utils.fc`

Description: Supported assets can be deprecated temporarily or forever. Such operation would be made at the level of the master contract. However, it could negatively impact methods at the level of the user contract where non-zero amounts could remain in the dict `user_principals` for deprecated keys. For instance, since all keys in `user_principals` are browsed, we could have in:

- 1. `account_health_calc()` a situation where the function returns `false` if one of the keys is not found in the dict `prices_packed`.
- 2. `check_not_in_debt_at_all()` a situation where the function returns `false` even if it is due to a debt in a deprecated token.
- 3. `is_liquidatable()` a situation where the function returns `false` if one of the keys is not found in the dict `prices_packed`, or where the function incorrectly considers the principal of a non-supported asset for the calculation of the liquidability of the account.
- 4. `get_avaliable_to_borrow()` a situation where the function returns `false` if one of the keys is not found in the dict `prices_packed`, or where the function incorrectly considers the principal of a non-supported asset for the calculation of the amount available to borrow.
- 5. `get_agregated_balances()` a situation where the function returns `false` if one of the keys is not found in the dict `prices_packed`, or where the function incorrectly considers the principal of a non-supported asset for the calculation of the aggregated balances.

Recommendation: Consider clarifying if all the items described above are working as expected or if the code should be updated. An option could be to send from the `master` contract the list of assets currently supported, so these 5 functions could ignore non-supported tokens. However, it should still be possible for users to withdraw deprecated assets. A clear plan should be defined to handle actions allowed for users when an asset is no longer supported in such a way that the solvency of the protocol cannot be affected.

EVAA-20

Each Muldiv Operation Could Explicitly Force a Round-up or Round-Down Based on the Context to Make the Behavior of the Code More Explicit and Controlled

• Undetermined ⓘ

Fixed

- ✓ Update

Explicit calls to `present_value_supply_calc()` or `present_value_borrow_calc()`, instead of `present_value_calc()`.
- ✓ Update

Marked as "Fixed" by the client.
Addressed in: `7a476629c4f8e2281d43f66235d219fb4214d215`, `33457a021c1c19c02ee953ba8aedfa938ada2dbc` and `bacac1761848a22c5cc3aad4b926cc9d5e764d44`.

Description: Muldiv operations round down by default. This could be wrong if it is not rounded in the interest of the protocol. For instance, amounts sent to the protocol should be rounded up, and the amount removed from the protocol should be rounded down. Also, depending on the context, computations done in `present_value_calc()` may require an opposite rounding direction depending on whether it corresponds to the present value of a borrowed amount or of a supplied amount. These particular operations could also be replaced by an explicit call to `muldiv()`. This can be particularly the case for nested `muldiv()` operations that can result in a loss of precision.

Recommendation: Consider adopting `muldiv()` operations wherever required (for instance in `present_value_calc()`), and adapting them to explicitly use the correct rounding direction.

EVAA-21 Usage of `impure` Specifier

• Undetermined ⓘ

Fixed

✓ Update

Three functions updated.

✓ Update

Marked as "Fixed" by the client.
Addressed in: `0c8ab44fe198daa7bad342c0aaf1f13ea0d75665`.

Description: As stated in the official [documentation of TON](#):

The ``impure`` specifier means that the **function** can have **some** side effects which cannot be ignored. **For** example, we should put an ``impure`` specifier **if** the **function** can modify contract **storage**, send messages, **or** throw an **exception when some data is invalid and the function is intended to validate** this data.

If `impure` is not specified and the result of the function call is not used, then the FunC compiler may and will **delete** this function call.

We identified that some functions in the codebase can revert but do not have an `impure` specifier. For instance:

- `master-withdrawal::master_core_logic_withdraw()` where there is a `throw_unless()` instruction;
- `universal-dict::upgrade_storage:get!()` where there is a `throw_unless()` instruction;
- `ton::cell_fwd_fee()` where there is a `throw_unless()` instruction;

Recommendation: Consider checking functions that can send messages, update the state, and throw (with a throw statement) or hard revert (due to an error) and add to them an `impure` specifier.

EVAA-22

Supply and Borrow Rates Might Be Outdated in Multiple Situations

• Undetermined ⓘ Fixed

- ✓

Update
Assets rates are now updated if they are older than 30 min. It reduces the impact of inconsistencies between prices.
- ✓

Update
Marked as "Fixed" by the client.
Addressed in: `7ea0860243c9263ec1d83987f15514f22ef5b03b` , `7c331ba487fd680dd2cafc52f81641b96e1ba554` .

File(s) affected: `master.fc`, `user-withdrawal.fc`

Description: The supply and borrow rates are updated at the beginning of all operations but only for the asset involved in the supply, borrow, or liquidation. Subsequent operations loop over all the user's assets to determine how much to borrow or if the user is liquidatable. However, these operations might use outdated rates, potentially leading to unexpected scenarios, such as a liquidatable position not being detected until it has already accrued bad debt.

For instance, in the liquidation flow, the rates are not updated before being passed to the user contract and, consequently, can be lower than expected by the liquidator. This may result in a failed liquidation since the liquidator is incentivized to liquidate the maximum possible amount, which may then exceed the maximum calculated by the user smart contract due to the lower rates.

Also, in the withdrawal process, within `master-withdrawal::master_core_logic_withdraw()` , the new supply and borrow rates for the withdrawn asset are calculated and sent to the user contract along with the **outdated** dynamic collections. When handling a full withdrawal, the function `calculate_maximum_withdraw_amount()` calculates the maximum amount that can be withdrawn, but it uses the old supply and borrowing rates from the outdated dynamic collections instead of the new rates to calculate `old_present_value` . Since the rates are updated only when a user interacts with a specific asset, there might be cases where there has not been any interaction for a significant time duration for the withdrawn asset. There could be two scenarios here:

- The rates for the withdrawn asset are very old: assuming the best scenario where the user has no debt, then the maximum amount would be a very outdated `old_present_value` .
- The rates for the other assets the user holds are very old: assuming the user has some debt, then the function `get_agregated_balances()` would return outdated balances (both positive and negative) which would also impact the correct amount the user should be able to withdraw.

Recommendation: We recommend checking the last time the rates were updated and update them if necessary.

EVAA-23 Missing Config Validation After Code Upgrade

• Undetermined ⓘ Fixed

- ✓

Update
Checks added with the function `master_admin::do_data_checks_process()` .
- ✓

Update
Marked as "Fixed" by the client.
Addressed in: `246f5a27675ad3a593a4385f8087fdaf98051a51` .

File(s) affected: `master.fc`

Description: After the master contract is updated, some configuration variables remain uninitialized. A subsequent transaction is expected to initialize them. However, the codebase does not verify whether these variables have been initialized before use. This oversight could lead to unexpected consequences in the accounting of unconfigured assets and potentially impact other funds, or even throw unexpected errors as described in [EVAA-2](#).

Recommendation: Consider adding some form of check that each asset's config both static and dynamic has been initialized.

EVAA-24 Production Readiness of the Codebase

• Undetermined ⓘ

Fixed

✓ Update

Listed issues were fixed.

✓ Update

Marked as "Fixed" by the client.
Addressed in: 70373fbf45f50569a428b54dd2e26679231a85ff .

Description: Here is a non-exhaustive list of items found that express that the codebase might not be production ready yet:

1. In `errors.fc` , comment includes `;;; NOTE: !!! I don't like this name: because of soft-checks, there are various reasons min_collateral might not be satisfied. Rename to "low_reward" or something similar`
2. In `fees.fc` , comments `;;; todo: measure and set values closer to practical usage, ";;; todo: Think more about it" and ";;; todo find exact value";`
3. In `master-storage.fc` , comment includes `;;;todo rm wallet_to_master everywhere";`
4. In `liquidate-message.fc` , comments `;;; todo: !!!! What info do the liquidators need in the reports?" and ";;; todo: maybe query_id?";`
5. In `master.fc` , comment indicating: `;; ----- todo: remove all of admin function below before opensource !!! ;`
6. In `user-upgrade.fc` , a part of the function `upgrade_user_process()` contains commented code.
7. In `master-withdrawal.fc` , the function `~dump()` is used;

Recommendation: Consider clarifying whether the Todos and the commented-out codes are necessary to keep or fix. However, since the codebase is planned to be further developed in the future, consider also adopting a versioning notation in the comments to remove any ambiguity and make sure that comments in the current code match the latest version of the code.

EVAA-25

Liquidators Are Incentivized to Perform More Smaller Liquidations to Get More Collateral Rewards

• Undetermined ⓘ

Acknowledged

i Update

Marked as "Acknowledged" by the client.

File(s) affected: `user-liquidate.fc`

Description: The liquidate process defines a maximum collateral reward capped at `50%` of the available collateral of the liquidated user for cases without bad debt. However, this threshold can be exceeded by initiating a second liquidation process if the user remains liquidatable. Therefore, a liquidator is incentivized to transfer assets amounting to slightly less than required, keeping the position unhealthy. If both liquidations are well coordinated the user cannot supply assets in-between, since his contract is locked for liquidations.

Exploit Scenario:

1. Assume a collateral factor of `60%` and a liquidation threshold of `70%` for all assets.
2. A user has a liquidatable position, having `100$` worth of collateral asset A and borrowing `80$` worth of asset B.
3. The liquidator could repay `50%` of the collateral, returning the position to a healthy state. However, he is more incentivized to limit this first operation to `30$` , leaving the user with `70$` of collateral and a loan of `50$` and allowing another liquidation.
4. The second liquidation targets `50%` of the collateral, leaving the user with `35$` worth of collateral overall, instead of `50$` . Note: this simplified example does not take the liquidation bonus into account.

Recommendation: Consider allowing a user to supply collateral during ongoing liquidation processes.

EVAA-26

[Self-Identified] Tracking Indexes Calculated Based on New Total Supply and Borrow

• Undetermined ⓘ

Fixed

✓ Update

Fixed as recommended.

✓ Update

Marked as "Fixed" by the client.
Addressed in: 6d9e0cf6941df751447fd6a638035b6168058701 .

File(s) affected: master-supply.fc

Description: *This issue was self-identified by the EVAA team during the audit and was added to the report. As such, its severity is Undetermined.*

Tracking indexes should be calculated based on old total supply / total borrow. However, the calculation is done based on new_total_supply on line 230 in master-supply.fc .

Recommendation: Calculate the tracking indexes based on the old total supply / total borrow.

EVAA-27

[Self-Identified] Repayments Can Be Blocked Due to the Current Max Cap Check

- Undetermined ⓘ

Fixed

✓ Update

Fixed as recommended.

✓ Update

Marked as "Fixed" by the client.
Addressed in: a39ca5acfcb68fcbacd1f85eb683d99c6a64d34a .

File(s) affected: master-supply.fc

Description: *This issue was self-identified by the EVAA team during the audit and was added to the report. As such, its severity is Undetermined.*

If a user makes a repayment and the max cap is lower than the current total supply, the code (on line 199 in master-supply.fc) will not let the user make a repayment because new_total_supply (which is just the old total_supply_principal + 0) will always be greater than max_token_amount .

Recommendation: Consider only preventing the new supply to go beyond the max cap, not the repaid amount.

EVAA-28

[Self-Identified] Global Tracking Indexes Not Updated when Updating Tracking Speed Config Parameters

- Undetermined ⓘ

Fixed

✓ Update

Fixed as recommended.

✓ Update

Marked as "Fixed" by the client.
Addressed in: 91eddfa3f5d9c1db120d133f85af9dfa1c2c13e7 and 9d9dfdaea13d2c1e2cf2428c2f2b49895ee80e97

File(s) affected: master-admin.fc

Description: *This issue was self-identified by the EVAA team during the audit and was added to the report. As such, its severity is Undetermined.*

Global tracking indexes tracking_supply_index , tracking_borrow_index and last_tracking_accrual should be updated during a configuration upgrade if the tracking speed is updated.

Recommendation: Consider updating the global tracking indexes tracking_supply_index , tracking_borrow_index and last_tracking_accrual during a configuration upgrade if the tracking speed is updated.

EVAA-29

[Self-Identified] Insufficient Validations of Messages Received From user.sc

- Undetermined ⓘ

Fixed

✓ Update

Fixed as recommended.



Update

Marked as "Fixed" by the client.

Addressed in: `b66cfee447c47063828f75ee78191b755123f924` .

File(s) affected: `master.fc`

Description: *This issue was self-identified by the EVAA team during the audit and was added to the report. As such, its severity is Undetermined.*

There are insufficient input validations on each message coming from user wallet (checks of addresses & custom_response_payload).

Recommendation: Consider checking the validity of addresses and custom_response_payload on each message coming from user wallet.

EVAA-30

[Self-Identified] Incorrect Message Body Parsed when Dealing with Reverted Operations

• Undetermined ⓘ

Fixed



Update

Fixed as recommended.



Update

Marked as "Fixed" by the client.

Addressed in: `9d9dfdaea13d2c1e2cf2428c2f2b49895ee80e97` .

File(s) affected: `master-revert-call.fc`

Description: *This issue was self-identified by the EVAA team during the audit and was added to the report. As such, its severity is Undetermined.*

In the contract `master-revert-call.fc` , function `revert_call_process()` , the slice `in_msg_body` is parsed instead of `revert_body` when `revert_op == op::supply_user` or `revert_op == op::liquidate_user` or `revert_op == op::idle_user` .

Recommendation: Consider parsing the slice `revert_body` instead of `in_msg_body` .

EVAA-31

[Self-Identified] Adding New Tokens Reverts Due to Incorrect Sequence of Operations

• Undetermined ⓘ

Fixed



Update

Marked as "Fixed" by the client.

Addressed in: `5ac05553e27afe7cc8585f84c4e0bff4e398e4cf` .

File(s) affected: `master-admin.fc`

Description: *This issue was self-identified by the EVAA team during the audit and was added to the report. As such, its severity is Undetermined.*

In `master-admin.fc` , the cumulated usage of the functions `add_new_token_dynamics_process()` , `update_master_lm_indexes()` and `do_data_checks()` can lead to a revert error when a token is added to the dict `asset_config_collection` before being added to the dict `asset_dyamics_collection` .

Recommendation: Consider adding in the same function a new token to these two collections.

EVAA-32 [Self-Identified] Supply Cap Check Can Be Bypassed

• Undetermined ⓘ

Fixed



Update

Marked as "Fixed" by the client.

Addressed in: `55096cf1fd091629ff8dad783f71fb4758eded46` .

File(s) affected: `master-supply.fc`, `user-supply.fc`

Description: *This issue was self-identified by the EVAA team during the audit and was added to the report. As such, its severity is Undetermined.*

Due to the asynchronous nature of TON, sending multiple supply requests simultaneously could lead to overriding the maximum supply cap check. Supply requests submitted to the master contract are processed one by one, before any consequential success, failure, or revert. As a result, the total supply value used in the user contract to enforce the maximum supply cap check only considers the updated total supply resulting from this single supply request, instead of considering all pending supply requests. This way, a malicious actor can supply more than the maximum supply cap and use it to borrow all the available assets.

Recommendation: Consider virtually accumulating into a variable of the master contract the amount of all pending supply requests, and passing that amount to the user contract. It will let the max supply cap check consider the scenario where all pending supply requests are successful. Then, deduce from this variable the virtual amount when the supply request either succeeds, fails, or reverts.

Auditor Suggestions

S1 Logic of Protocol Reserves Should Be Clarified

Acknowledged

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

It will be documented

Description: During the audit, the client indicated that protocol reserves should be used as follows:

- it can be used to withdraw a supplied amount;
- it can be used during a liquidation;
- it cannot be borrowed; These three aspects could be more documented in the code.

Recommendation: Consider clarifying how protocol reserves should be managed.

S2 Redundant Call of `around_zero_split()`

Fixed

Update

Fixed as recommended.

Update

Marked as "Fixed" by the client.
Addressed in: `be453d2a9a31f4e0746dbfb6fefff6a653837674` .

File(s) affected: `user-withdrawal.fc`

Description: In the function `user-withdrawal::withdraw_user_process()` , when the borrow is collateralized and `borrow_amount_principal <= 0` , the function `around_zero_split()` is called twice unnecessarily.

Recommendation: We recommend moving the second call inside the `if (borrow_amount_principal > 0)` .

S3 Usage of Magic Numbers

Fixed

Update

Constants used instead.

Update

Marked as "Fixed" by the client.
Addressed in: `5291ab5f5dea1b8250f331caec5e882a66668274` .

File(s) affected: `prices-packed.fc`

Description: Magic numbers should be avoided. For instance:

1. In the function `prices-packed::parse_check_oracles_data()`, it is unclear what the `3` means in `throw_unless(error::prices_incorrect_proof, ps~load_uint(8) == 3);`
2. The freeze time for new upgrades is currently checked to be at least `30` seconds by comparing it to a magic value. Instead, this value may be part of the storage, so that it can be changed. Also, it's important for the community to check the new code of the contract, a process that will take more than `30` seconds.

Recommendation: Consider using constants or storage variables instead of hardcoded numerical values.

S4 Typos Identified

Fixed

✓ Update

Listed issues fixed.

✓ Update

Marked as "Fixed" by the client.

Addressed in: `7e674e8e8304aa5a7642957ad5b74a4bcaeb8755`.

Description: Here is a list of typos identified:

1. In `constants.fc`, in the name of `constants::reserve_liquidatoin_scale`;
2. In `user-utils.fc`, in the name of function `get_avaliable_to_borrow()`;
3. In `master-admin.fc`, in the names of `new_assets_dynamics` and `last_accural`;
4. In `master-utils.fc`, in the name of `accure_interest()`;

Recommendation: Correct the typos.

S5 Implicit Code Imports

Fixed

✓ Update

Fixed as recommended.

✓ Update

Marked as "Fixed" by the client.

Addressed in: `ccd4ff8b07426c0ae10662577b9ac0b4effd235c`.

Description: It is recommended to include all files used by the current file, in order to improve the code readability. For example, the function `master-other::get_store_process()` calls `send_message()`, but the source of the implementation is unclear. The codebase compiles because the inline keyword instructs the compiler to copy the function's body into the calling function within `master.fc`, which includes `logic/tx-utils.fc` where `send_message()` is defined. A similar case occurs in:

1. `user-supply::supply_user_process()` where `present_value()`, `principal_value()`, and `around_zero_split()` are defined in `logic/utils.fc` which is not included in `user-supply.fc`.
2. `asset-config-packer::asset_config_collection::decimals()` where `load_address_hash()` is used without including `basic-types.fc`.

Recommendation: Consider including all files used by the current file, in order to improve the code readability.

S6 Functions Not Used or Identical

Fixed

✓ Update

Functions removed or code refactored.

✓ Update

Marked as "Fixed" by the client.

Addressed in: `ad3a5698d918c4266c419f379702a4ca49c42218`.

File(s) affected: `user-utils.fc`, `master-utils.fc`, `utils.fc`

Description: 1. In `user-utils.fc`, the functions `get_account_asset_balance()` and `get_present_value()` are identical;
2. In `user-utils.fc`, the function `enough_balance_token()` is not used;
3. In `master-utils.fc`, the function `accure_interest()` is not used;

4. In `utils.fc`, the functions `calc_supply_principals()` and `calc_withdraw_principals()` are not used;

Recommendation: Consider refactoring or removing these functions.

S7 Suggested Rewording

Fixed



Update

Fixed as recommended.



Update

Marked as "Fixed" by the client.

Addressed in: `f48ae254dbcbf334a2503befe32b3013b97d68e9`.

File(s) affected: `withdraw-message.fc`, `user-supply.fc`, `master-supply.fc`

Description: 1. In `withdraw-message.fc`, the parameter `specific_error_code` of `pack_withdraw_excess_message_with_data()` could be renamed since it can be called with the value `op::withdraw_success`;

2. In `user-supply.fc`, there is an incorrect comment `;; Update user_principals and Unlock` in function `supply_unlock_fail_process()` since we do not update `user-principals` if a supply request fails;

3. In `master-supply.fc`, the names `supply_success_revert_user` and `fee::supply_success_revert_user` suggest that the operation is related to a revert;

Recommendation: Consider rewording these items.

S8 Usage of Nested Storage

Acknowledged



Update

Marked as "Acknowledged" by the client.

Description: Consider using nested storage wherever relevant, as described [here](#).

Recommendation: Consider assessing if additional nested storage should be added or not.

S9 Unused Function Parameters

Acknowledged



Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

`op` is for simple identification of the business sense of transaction. `query_id` is important to get which chain of transactions a particular transaction belongs to (given chain of transactions may begin in third party protocol, and ends in another third-party protocol, as a simple example: combination of liquidation and swap)

Description:

1. In several contracts, use empty parameter names if these are not used in the function.
2. Remove function parameters if they are not used. For instance, for the numerous function calls of `master.fc` and `user.fc`.
3. In `withdraw-message.fc`, none of the parameters `op` and `query_id` are used in `pack_withdraw_success_excess_message()`;
4. In `supply-message.fc`, none of the parameters `op` and `query_id` are used in `pack_supply_success_excess_message()`;
5. In `liquidate-message.fc`, none of the parameters `op` and `query_id` are used in `pack_liquidate_excess_message()`;

Recommendation: Consider refactoring these functions wherever relevant.

S10 Duplicated Code

Fixed



Update

Code refactored.



Update

Marked as "Fixed" by the client.
Addressed in: 6d9e0cf6941df751447fd6a638035b6168058701 , 954da8bed750fac6d880d55557fcc33ec7c074c1 .

Description: The following snippet of code is duplicated at several locations and may be added to a single function:

```
if (now() - last_tracking_accural > 0) {
  int timeElapsed = now() - last_tracking_accural;
  if (new_total_supply >= min_principal_for_rewards) {
    tracking_supply_index += muldiv(base_tracking_supply_speed * timeElapsed, fast_dec_pow(decimals),
new_total_supply);
  }
  if (new_total_borrow >= min_principal_for_rewards) {
    tracking_borrow_index += muldiv(base_tracking_borrow_speed * timeElapsed, fast_dec_pow(decimals),
new_total_borrow);
  }
  last_tracking_accural = now();
}
```

Recommendation: Consider this suggestion.

S11

Possible Locked Funds if Upgraded Contracts Become Not Upgradeable and Have No Way to Rescue Funds

Fixed

- ✓

Update
There is now a way to prevent funds being locked.
- ✓

Update
Marked as "Acknowledged" by the client.
The client provided the following explanation:

Actually this is solved. the finding contradicts to the EVAA-1 finding. our approach in EVAA-1 – we keep overpower, but decentralise access to it, so we can save funds even if store is lost

File(s) affected: master-storage.fc

Description: The code of the master and user contracts can be updated via the function set_c3(). They can hold funds. If an upgrade is made, the contract holds funds, and the new version does not have a way to move the funds or update again the code, funds could remain locked in the contract. The impact is high but the likelihood of this scenario is unlikely, so the severity is considered Low.

Recommendation: Consider making sure that, if an update of the code of the contract is submitted (master or user), the new contract still has an upgradeable feature or, if not, no funds could remain blocked after this update.

S12

[Self-Identified] Looping over Provided Prices Is Gas Expensive Can Be Done only Once in calculate_maximum_withdraw_amount()

Fixed

- ✓

Update
Marked as "Fixed" by the client.
Addressed in: 4017c2f2ab19f39e86ed01989450478593189b14 .

File(s) affected: user-utils.fc

Description: This code simplification was self-identified by the EVAA team during the audit and was added to the report.

The call to get_agregated_balances() in the function calculate_maximum_withdraw_amount() was no longer required after the first fixes since the parameters returned by get_agregated_balances() were no longer used and the fact that the dict prices_packed contains a valid price for all assets borrowed by the user was also checked later in the same function, making this call redundant. Since this operation is gas expensive, removing it would save gas.

Recommendation: Consider removing this call to get_agregated_balances() in order to save gas.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not post an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

- 935...bf2 ./contracts/user.fc
- cf0...323 ./contracts/blank.fc
- 8f7...2a3 ./contracts/master.fc
- 110...b5a ./contracts/data/universal-dict.fc
- f70...908 ./contracts/data/basic-types.fc
- 96c...401 ./contracts/data/prices-packed.fc
- 3ec...9d1 ./contracts/data/asset-dynamics-packer.fc
- 60a...d51 ./contracts/data/asset-config-packer.fc
- 632...28e ./contracts/external/stdlib.fc
- 322...30e ./contracts/external/openlib.fc
- 4de...abc ./contracts/external/ton.fc
- 6a6...2a4 ./contracts/storage/user-storage.fc
- 564...452 ./contracts/storage/master-storage.fc
- 3c7...acd ./contracts/storage/master-upgrade.fc
- a42...149 ./contracts/storage/user-upgrade.fc
- 2b7...d2f ./contracts/logic/user-revert-call.fc
- 5d4...a2b ./contracts/logic/addr-calc.fc
- bce...b2f ./contracts/logic/tx-utils.fc
- 8fb...4f2 ./contracts/logic/user-get-methods.fc
- f10...572 ./contracts/logic/master-get-methods.fc
- 3e9...716 ./contracts/logic/master-if-active-check.fc
- 702...2b3 ./contracts/logic/user-upgrade.fc
- 8b2...771 ./contracts/logic/utils.fc
- 37d...4c1 ./contracts/logic/user-utils.fc
- 488...8ac ./contracts/logic/master-utils.fc
- d9c...437 ./contracts/constants/constants.fc
- 116...8b2 ./contracts/constants/fees.fc
- 762...fe3 ./contracts/constants/errors.fc
- 8e0...b35 ./contracts/constants/op-codes.fc
- 03a...95c ./contracts/constants/logs.fc
- e0f...49f ./contracts/core/user-other.fc

- d96...6c5 ./contracts/core/user-withdrawal.fc
- d73...407 ./contracts/core/master-other.fc
- 14a...12f ./contracts/core/user-supply.fc
- f5c...857 ./contracts/core/user-liquidate.fc
- 1a3...cb4 ./contracts/core/master-revert-call.fc
- 61d...3b8 ./contracts/core/master-liquidate.fc
- 696...865 ./contracts/core/master-supply.fc
- cb2...d24 ./contracts/core/user-admin.fc
- 816...7be ./contracts/core/master-admin.fc
- 090...c8a ./contracts/core/master-withdrawal.fc
- 8aa...35e ./contracts/messages/withdraw-message.fc
- 6a2...c83 ./contracts/messages/liquidate-message.fc
- 1e9...0cb ./contracts/messages/upgrade-header.fc
- af1...2d9 ./contracts/messages/admin-message.fc
- c87...584 ./contracts/messages/supply-message.fc
- ad7...65e ./contracts/messages/idle-message.fc

Automated Analysis

N/A

Test Suite Results

All tests passed, happy and unhappy paths are covered. Still, the test suite contains some "todo" statements.

Fix review update

The team added some tests, and all passed, happy and unhappy paths are covered. Still, the test suite contains some "todo" statements.

Test "Enable-Disable" passed

Test "Supply - TON - Cross Zero" passed

Test "Supply - TON Single, but master sc is off" passed

Test "Withdraw - No signature" passed

Test "Supply - Jetton - Cross Zero" passed

Test "Supply - TON Single" passed

Test "Withdraw - Bad signature" passed

Test "Withdraw - Old signature" passed

Test "Withdraw - Barely OK dated signature" passed

Test "Withdraw - Future signature" passed

Test "Withdraw - No prices" passed

Test "Withdraw - Short signature" passed

Test "Reclaim - TON - Good" passed

Test "Reclaim - TON - More" passed

Test "Reclaim - TON - Not enough Liquidity" passed

Test "Reclaim - Jetton - Good" passed

Test "Reclaim - Jetton - More" passed

Test "Reclaim - Jetton - Not enough Liquidity" passed

Test "Borrow - TON - Good" passed

Test "Borrow - TON - Not Collateralized" passed

Test "Borrow - TON - Not enough Liquidity" passed

Test "Borrow - TON - Zero Collateral" passed

Test "Borrow - TON - Zero Liquidity" passed

Test "Borrow - Jetton - Good" passed

Test "Borrow - Jetton - Not Collateralized" passed

Test "Borrow - Jetton - Not enough Liquidity" passed

Test "Borrow - Jetton - Zero Collateral" passed

Test "Borrow - Jetton - Zero Liquidity" passed

Test "Borrow - TON and Jetton - Good" passed

Test "Withdraw - TON - Tiny amount" passed

Test "Withdraw - Jetton - Tiny amount" passed

Test "Withdraw - TON - Cross Zero" passed

Test "Liquidate - Jetton With TON - Basics" passed

Test "Liquidate - TON With Jetton - Basics" passed

Test "Liquidate - Jetton With Jetton - Basics" passed
Test "<Cap> Supply - TON Single" passed
Test "<Cap> Supply - TON Single edge" passed
Test "<Cap> Supply - TON Single over cap edge" passed
Test "<Cap> Supply - TON Single over cap" passed
Test "<Cap> Supply - TON Double" passed
Test "<Cap> Supply - TON Double edge" passed
Test "<Cap> Supply - TON Double over cap" passed
Test "<Cap> Supply - Jetton Single" passed
Test "<Cap> Supply - Jetton Single edge" passed
Test "<Cap> Supply - Jetton Single over cap edge" passed
Test "<Cap> Supply - Jetton Single over cap" passed
Test "<Cap> Supply - Jetton Double" passed
Test "<Cap> Supply - Jetton Double edge" passed
Test "<Cap> Supply - Jetton Double over cap" passed
Test "<Cap> Supply - Multiple edge" passed
Test "Supply - Jetton Single" passed

Test summary:

51/51 tests passed. 0 tests failed

Fix review update

Test "Enable-Disable" passed
Test "Do data checks" passed
Test "Supply - TON - Cross Zero" passed
Test "Supply - Jetton - Cross Zero" passed
Test "Supply - Jetton Single" passed
Test "Supply - TON Single, but to bad addresses" passed
Test "Supply - TON Single, but master sc is off" passed
Test "Supply - Jetton Single, but to bad addresses" passed
Test "Supply - Custom payload tests" passed
Test "Withdraw - No signature" passed
Test "Withdraw - Bad signature" passed
Test "Withdraw - Old signature" passed
Test "Withdraw - Barely OK dated signature" passed
Test "Withdraw - Future signature" passed
Test "Withdraw - Bad destinations" passed
Test "Withdraw - Short signature" passed
Test "Reclaim - TON - Good" passed
Test "Withdraw - No prices" passed
Test "Reclaim - TON - More" passed
Test "Reclaim - TON - Not enough Liquidity" passed
Test "Reclaim - Jetton - Good" passed
Test "Reclaim - Jetton - More" passed
Test "Reclaim - Jetton - Not enough Liquidity" passed
Test "Borrow - TON - Good" passed
Test "Borrow - TON - Not Collateralized" passed
Test "Borrow - TON - Not enough Liquidity" passed
Test "Borrow - TON - Zero Collateral" passed
Test "Borrow - TON - Zero Liquidity" passed
Test "Borrow - Jetton - Not enough Liquidity" passed
Test "Borrow - Jetton - Not Collateralized" passed
Test "Borrow - Jetton - Good" passed
Test "Borrow - Jetton - Zero Collateral" passed
Test "Borrow - Jetton - Zero Liquidity" passed
Test "Borrow - TON and Jetton - Good" passed
Test "Withdraw - TON - Tiny amount" passed
Test "Withdraw - Jetton - Tiny amount" passed
Test "Borrow - Custom payload tests" passed
Test "Withdraw - Custom payload tests" passed
Test "Withdraw - TON - Cross Zero" passed
Test "Reserve factor test" passed
Test "Liquidate - Jetton With Jetton - Basics" passed
Test "Liquidate - TON With Jetton - Basics" passed
Test "<Cap> Supply - TON Single" passed
Test "<Cap> Supply - TON Single edge" passed
Test "Liquidate - Jetton With TON - Basics" passed
Test "<Cap> Supply - TON Single over cap edge" passed
Test "<Cap> Supply - TON Double" passed
Test "<Cap> Supply - TON Double edge" passed


```
Test "<Cap> Supply – TON Double over cap" passed
Test "<Cap> Supply – Jetton Single" passed
Test "<Cap> Supply – Jetton Single edge" passed
Test "<Cap> Supply – Jetton Single over cap edge" passed
Test "<Cap> Supply – Jetton Single over cap" passed
Test "<Cap> Supply – TON Single over cap" passed
Test "<Cap> Supply – Jetton Double" passed
Test "<Cap> Supply – Jetton Double edge" passed
Test "<Cap> Supply – Jetton Double over cap" passed
Test "<Cap> Supply – Multiple edge" passed
Test "Supply – TON Single" passed

-----

Test summary:
59/59 tests passed. 0 tests failed
```

Code Coverage

N/A - No feature currently exists to measure the code coverage of a FunC codebase

Changelog

- 2024-07-16 - Initial report
- 2024-10-10 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 500 audits and secured over \$200 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Polygon, Avalanche, Curve, Solana, Compound, Lido, MakerDAO, Arbitrum, OpenSea and the World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Near, Flow, Avalanche, Solana, Cardano, Binance Smart Chain, Hedera Hashgraph, Tezos
- DeFi: Curve, Compound, Maker, Lido, Polygon, Arbitrum, SushiSwap
- NFT: OpenSea, Parallel, Dapper Labs, Decentraland, Sandbox, Axie Infinity, Illuvium, NBA Top Shot, Zora
- Academic institutions: National University of Singapore, MIT

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites&aspo; owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no

responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and and may not be represented as such. No third party is entitled to rely on the report in any any way, including for the purpose of making any decisions to buy or sell a product, product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or or any open source or third-party software, code, libraries, materials, or information to, to, called by, referenced by or accessible through the report, its content, or any related related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

